

Exploring the Trade-offs between Energy and Performance of Federated Learning Algorithms

Mai Huong Do

July 21, 2025

This report presents my activity as part of my thesis, directed by **Georges Da Costa** and **Millian Poquet**, within the **SEPIA team**.

Contents

1	Introduction	4
1.1	Thesis context	4
1.2	Objective and plan	4
2	Thesis Progress	4
2.1	Last year - 1st year	4
2.2	Current year - 2nd year	4
2.2.1	Motivation and Contribution	5
2.2.2	Related work	5
2.2.3	Framework description	5
2.2.4	Experiments	9
2.3	Future work - 3rd year	16
3	Work situation	17
3.1	Difficulties encountered	17
3.2	Research Outputs	17
3.3	Teaching	17
3.4	Training progress	17

1 Introduction

1.1 Thesis context

Federated Learning (FL) is an emerging distributed ML paradigm where nodes compute local gradients and share them with a central server that coordinates training. This approach allows nodes (e.g., IoT devices, smartphones) to train models collaboratively while keeping data local, thus addressing privacy and legal concerns.

However, FL introduces new challenges beyond traditional ML, including: (i) communication and energy overhead, (ii) longer training due to non-iid data, and (iii) unbalanced workloads across heterogeneous devices. Energy efficiency remains largely overlooked in recent FL variants such as cross-device FL, cross-silo FL, federated transfer learning, and personalized FL.

The ANR project DELIGHT (started in early 2023) aims to integrate energy efficiency as a key metric in FL to promote sustainable learning. This position will address the first challenge using the Flower framework.

1.2 Objective and plan

Our team contributes to the DELIGHT project by modeling energy consumption, optimizing hybrid CPU/GPU usage, and developing middleware for clusters, mesh, and cloud environments.

My PhD focuses on exploring energy-performance tradeoffs in FL, proposing new energy estimation methods, and creating models and libraries for measuring energy at various levels (data, hardware, software, network). I aim to leverage framework parameters like floating-point precision and data transfer strategies, with use cases in speech recognition or object detection.

Phases of the PhD:

- Set up an experimental environment on Grid'5000 (g5k) to gather performance and energy metric.
 - Create a use-case for the Flower framework.
 - Build a reproducible and automated framework to obtain metrics for this use case.
- Propose, model, and implement the different leverages.
- Explore the impact of leverage on both energy and performance. **(current status)**

2 Thesis Progress

2.1 Last year - 1st year

From the beginning to the end of August 2024, I:

- Conducted a survey of related work.
- Carried out initial experiments on energy measurement in a distributed system (Grid'5000).

The future work outlined last year included:

In the next steps, I plan to continue building a reproducible and automated framework, with a focus on improving stability when using Expetator. I also aim to propose, model, and implement various control strategies to evaluate their impact on energy and performance. Beyond the current FL framework, I intend to test alternative frameworks or newer versions. Additionally, I will deepen my understanding of prior works on energy modeling in FL, which I have already surveyed, to support my own contributions.

2.2 Current year - 2nd year

As planned, I have since developed FedE-ator, a framework for measuring energy consumption of FL in distributed systems. The details of its architecture and capabilities will be discussed in the next sections.

2.2.1 Motivation and Contribution

Understanding FL’s energy behavior is key to improving its sustainability. Existing energy measurement methods fall into two categories:

- **Hardware-based**, using external tools like wattmeters.
- **Software-based**, using system tools (e.g., Mojito/S¹, Perfetto², PowerAPI³, Alumet⁴) to estimate component-level energy.

However, most tools target general-purpose systems. FL adds complexity due to its distributed nature and frequent communication. Hardware-only tools lack insight into component-level usage, while software-only tools may miss total energy costs.

To overcome this, I propose a hybrid monitoring framework combining both approaches for more accurate and granular FL energy analysis.

Our contribution consists of developing a framework (FedE-ator) with the functions below:

- Measure energy consumption for FL processes in distributed systems (Grid’5000/SLICES-FR), with synchronized use of both hardware and software monitoring methods.
- Support for controlling CPU and GPU operating frequencies (using DVFS), enabling energy-performance tradeoff studies.
- Support for running multiple repetitions in a reproducible way to improve the reliability of experimental results.

The remainder of this work is organized as follows. Section 2.2.2 presents related studies concerning energy measurement in FL. Section 2.2.3 describes the FedE-ator framework in detail. Section 2.2.4 presents the experimental validation of the framework with CPUs, GPUs nodes and large-scale set up.

2.2.2 Related work

To assess the state of energy measurement in FL, I explored prior work using two approaches:

First, I reviewed recent studies (2020–2025) explicitly addressing energy in FL. Most were limited: [SRKB22] focused only on modeling; [KNW⁺23] measured battery usage via a mobile app on smartphones; and [MHdBPM25] used simulation with no real measurements.

Second, I examined broader FL studies reporting energy metrics. I found six works using diverse tools: hardware wattmeters (Energon, Monsoon, Pioghax), simulation (Vessim), and profilers (Perfetto, Trepn, Dstat). However, these tools often targeted specific platforms, lacked synchronization, or were not generalizable.

A key challenge is synchronizing energy measurements across distributed clients. Unlike single-device ML, FL’s decentralized nature complicates alignment with training or communication phases. Manual coordination is error-prone, yet no prior work proposes a robust or reproducible methodology.

In short, energy measurement in FL is still an open challenge, with no standard or transparent approach available for distributed environments.

2.2.3 Framework description

FedE-ator was developed for use in Linux-based distributed systems. For evaluation purposes, I propose an implementation using a specific wattmeter plugin for Grid’5000⁵. This platform is a large-scale and flexible testbed for experiment-driven research in all areas of computer science, with a focus on parallel and distributed computing, including Cloud, HPC, Big Data, and AI.

The standard FL process usually follows these steps:

- **Step 1 - Initialization:** The central server initializes a global model and sends it to all participating clients.

¹<https://gitlab.irit.fr/sepia-pub/mojitos>

²<https://perfetto.dev/>

³<https://powerapi.org/>

⁴<https://alumet.dev/>

⁵<https://www.grid5000.fr/>

- **Step 2 - Local Training:** Each client uses its own local dataset to train the model independently for one or a few times (called local training epochs).
- **Step 3 - Upload Updates:** After local training, each client sends its model updates (*e.g.*, gradients or new model weights) back to the server.
- **Step 4 - Aggregation:** The server collects updates from multiple clients and aggregates them (using aggregation strategies) to update the global model.
- **Step 5 - Repeat:** Steps 2–4 are repeated for multiple rounds until the model converges or reaches an early stopping point.

Mapping FL into distributed system infrastructure:

- Each client and central server is deployed on a separate compute node.
- Communication between server and clients is handled over the distributed system network (Grid’5000 internal network⁶).

Grid’5000 provides the realistic environment that support the deployment and evaluation of FL experiments. It offers a precise configuration of experimental environments. Additionally, its distributed infrastructure across multiple clusters enables flexible placement of clients and the server, making it possible to reflect different network conditions.

A. Components and functions of FedE-ator

1. Configuration module (JSON) allows users to define a list of instances. Each instance includes:

- Hardware settings (*e.g.*, CPU, GPU specifications)
- FL framework settings (*e.g.*, server/clients configuration, execution commands and IP addresses)

2. Controller module reads the **Configuration** and orchestrates the experiment. An experiment involves executing through each setting of all **instances**, each executed with a defined number of repetitions. Each **instance** can be executed at different frequencies as specified. A single execution is referred to as a **RUN**, which consists of the following sub-processes:

- FL process: initiated by the **FL module**.
- Monitoring processes: initiated by the **Monitoring module**, which continuously observes the FL process from start to finish.

3. FL module uses **SSH** on Grid’5000 to access nodes assigned as servers and clients, activate the FL commands, and record the logs to the output directory.

4. Monitoring module uses Expetator⁷, an extensible orchestration tool for running and monitoring HPC experiments on Grid’5000. It supports both Mojito/S and Kollect⁸ - two components for energy measurement.

- Mojito/S monitors processor and memory power consumption (via RAPL), as well as system and network metrics. It is deployed on each node, with deployment controlled by Expetator.
- Kollect collects power consumption data of whole nodes from the Grid’5000 external wattmeters.

B. Energy consumption measurement I used both software profiling and hardware-based solutions to measure energy. Two measurement solutions are synchronized by timestamps, from begin to finish of FL process.

Fig. 1 provides details of the energy monitoring module, including all measurement methods within Kollect and Mojito/S.

⁶<https://www.grid5000.fr/w/Grid5000:Network>

⁷<https://expetator.readthedocs.io/>

⁸https://www.grid5000.fr/w/Monitoring_Using_Kollect

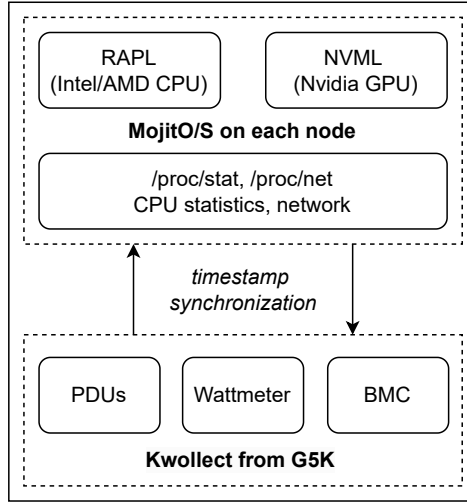


Figure 1: FedE-ator energy monitoring module

1. Software profiling solution - Mojito/S Mojito/S is an energy and network monitoring software at the O/S level, runs on GNU/Linux [dC21]. The unit of energy is in micro-Joules and the network packet size is in bytes. Mojito/S monitoring processes are deployed on all nodes and launched using the **mpirun** command from the Open MPI library⁹. Mojito/S supports extracting performance information from various system sources, including the **/proc** directory, Running Average Power Limit (**RAPL**), NVIDIA Management Library (**NVML**), hardware performance counters, and others.

- **/proc/stat**: holds various pieces of information about the kernel activity and is available on every Linux system. From this file I get CPU usage statistics of Linux Kernel.
- **/proc/net**: provides a comprehensive look at various networking parameters and statistics. It describes aspects of the system's network configuration. Mojito/S allows automatic detecting the type of network configuration and gets network information (*e.g.*, number/size of packets received/transmitted).

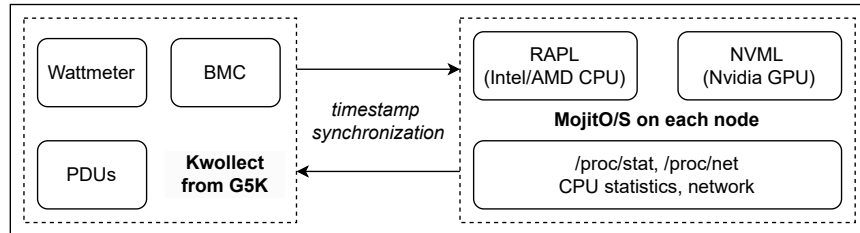


Figure 2: RAPL domains and corresponding CPU components. Components not in bold do not form a domain by themselves. A "CPU die" refers to a small block of semiconducting material on which the processor's functional circuits (*e.g.*, cores, caches) are fabricated.

- **RAPL**: is a feature which allows monitoring energy consumption across different domains of the CPU chip, attached DRAM and on-chip i-GPU [Int64]. This feature is integrated in modern x86 CPUs from Intel and AMD. Fig. 2 show the power domains supported by RAPL in a CPU.

In addition to the domains above, some literature [RT24, KHN⁺18] mentions **psys** as a measurement for the entire machine, closely matching power meter readings. It includes the whole SoC, covering the CPU, GPU, System Agent, PCH, eDRAM, and other domains. However, it only applies to certain models, such as some Skylake versions. For our current work, I am not utilizing **psys**.

⁹<https://www.open-mpi.org/>

- **NVML**: is a C-based programmatic interface for monitoring and managing various states within NVIDIA GPUs (*i.e.*, GPU core utilization, memory utilization, power consumption, temperature), is also the underlying library for the NVIDIA-supported `nvidia-smi` tool¹⁰.

Regarding power consumption, NVML provides the current total power usage of the graphics card, encompassing the GPU core, memory, and other on-board components.

2. Hardware-based solution - Kwollect On Grid’5000, energy consumption is measured using plug-in devices such as wattmeters, Board Management Controllers (BMCs), and Power Distribution Units (PDUs). The recorded power values are expressed in Watts. Kwollect stores the measured data, and Expetator collects it through the Grid’5000 API.

C. Additional features

1. DVFS Our framework includes a DVFS adjustment function that allows controlling CPU/GPU frequencies to analyze their impact on FL performance and energy consumption. Before executing each instance, the **Monitoring module** sets the frequency of all CPUs/GPUs as specified in the **Configuration**. After execution (*i.e.*, when the FL process is complete), it restores the frequencies to their default values.

This function detects the specified frequencies and iterates through them one by one. It supports three configuration options:

- $f = f_{max}$: Maximum performance mode (set the frequency to the maximum and execute the **RUN**).
- $f = \{f_{min}, f_{max}\}$: Minimum and maximum performance modes (first set the frequency to the minimum and execute the **RUN**, then switch to the maximum and repeat the **RUN**).
- $f = \text{list}[f_i]$: Custom frequency selection (for each value in the list, set the frequency and execute the **RUN**, then move to the next value and repeat).

This function enables frequency tuning within the system’s available configuration range.

2. Multiple instances and repetition A single **Configuration** can include one or multiple **instances** with different FL and hardware (DVFS) settings. Each **instance** is identified by an ID number, a name, and an output folder for simplified result management. The framework also allows specifying the number of repetitions, enabling multiple runs within a single experiment without requiring separate experiment restarts.

3. Configuration for FL framework flexibility

- **Flexible settings within a single FL setting**: Within a given FL framework (*e.g.*, Flower), users can scale the experiment to involve multiple clients (*e.g.*, 1, 2, or more) by modifying the client setting list in the **Configuration**. For instance, to evaluate scenarios with 2 and 10 clients in a single experiment, define two **instances**: the first includes a client setting list with two client configurations; the second scales up by including eight additional client configurations to form a total of 10.
- **Flexibility across multiple FL settings**: Our FedE-ator supports flexible FL-related settings, executing several frameworks with a single configuration file. Each **instance** in a **Configuration** can be associated with a different framework and customized accordingly. To achieve this:
 - Executing experiments using different FL frameworks (*e.g.*, Flower, PySyft) or customizing FL parameters (*e.g.*, model aggregation strategies, hyperparameters, dataset) by preparing the corresponding execution scripts of the FL frameworks.
 - Updating the command and argument fields in each instance to match the execution logic of the FL framework.

¹⁰<https://docs.nvidia.com/deploy/nvml-api/index.html>

This modularity enables the launching of extensive campaigns covering a wide range of FL scenarios with minimal manual effort.

2.2.4 Experiments

I conducted several experiments to validate the usability of our framework for each scenario:

- **CPU nodes:** I used four CPU nodes (one server and three clients) across four CPU frequency settings, with four FL configurations (comprising two server strategies and two client models) on the CIFAR-10 dataset. Each configuration combination was executed twice.
- **Large-scale:** I used 20 GPU nodes (one server and 19 clients) across four CPU frequency settings, with one FL configuration on the CIFAR-10 dataset. Each configuration was performed once.
- **GPU nodes:** I used four GPU nodes (one server and three clients) at maximum GPU frequency settings, with two FL configurations (two options for local training) on the CIFAR-10 dataset. Each configuration was performed once.

Our experiment was designed with openness and reproducibility in mind, the code and data of our work are available here [Hid]. Details of each experiment are shown below.

A. Validating framework usability on CPU nodes

1. Experimental setup I executed this experiment on four machines from the **taurus** cluster on the Grid’5000 testbed. This cluster was selected because it contains Intel CPUs with a feature of RAPL and the external power meter (wattmeters). List of used nodes:

- Server node: **taurus-1**.
- Client nodes: **taurus-8, taurus-9, taurus-13**.

Detailed specifications of these machines are as follows:

- Model: Dell PowerEdge R720.
- Manufacturing/arrival date: 2012-07-16 / 2012-09-14.
- CPU: Intel Xeon E5-2630 (Sandy Bridge), x86_64, 2.30GHz, 2 CPUs/node, 6 cores/CPU.
- Memory: 32 GB.
- Storage: 299 GB HDD RAID-0, controller: Dell PERC H710.
- Network: **eth0/enp68s0f0**, Ethernet, configured rate: 10 Gbps, model: Intel Ethernet 10G 2P X520 Adapter, driver: **ixgbe**, SR-IOV enabled.

The FedE-ator framework requires Python v3.9 or higher and Expetator v0.35. Our experiments utilize Flower v1.13, a FL framework. The required dependencies for running the example benchmark include flwr-dataset, TensorFlow, NumPy, and Scikit-learn.

A FL strategy is a method used to combine ML models from multiple clients into a central server. In our benchmark, I use the Flower framework [BTM+20] because it supports heterogeneous clients, allowing participants to use different devices (CPUs, GPUs) and machine learning libraries (*e.g.*, TensorFlow, PyTorch) without strict compatibility requirements. I apply two FL strategies: **FedAvg** [RCZ+20] and **FedAvgClients**. **FedAvg** is a common approach that aggregates updates from all available clients in each round. In contrast, **FedAvg2Clients** is a variant that randomly selects a subset (*i.e.*, 67% of available clients) to participate in training in each round, thereby reducing the communication and computation cost per round. Non-selected clients are idle during that round and do not perform any training.

On each client device, training occurs in multiple small steps called epochs, which represent how many times the model goes through the local dataset. For each round of FL training, a client trains its model for a number of epochs, chosen from the set $\{1, 2\}$. Other details are provided in Table 1.

Based on the hardware conditions of **taurus**, the chosen CPU frequencies for this experiment are [1.2, 1.6, 2, 2.3] GHz, which includes f_{min} , f_{max} , and two available frequencies in between.

Table 1: Details of the FL setup used in our experiments: task, model, dataset, stopping rounds and client data partitioning.

ML task	image classification
Model	MobileNetV2, batch size=32
Stopping round	15
Dataset	CIFAR-10 (32×32 RGB images of vehicles and animals, 10 classes, 60000 samples)
Data distributed	train 80%, test 20%
Data partition	divide the full dataset equally among the number of clients

2. Results analysis I present the initial version of our results analysis here. Figures 3 to 6 show the visualizations of raw data from a single run in a specific setting (model aggregation: **FedAvg**, dataset: CIFAR-10, 1 local training epoch, CPU frequency: 1.6 GHz) to demonstrate the outputs collected by our framework. I then perform an analysis of the remaining measured data to extract insights. The descriptions of each figure are provided below.

Fig. 3 presents the energy usage over time in each node, broken down by domain (package, core, and DRAM). The ability to track fine-grained domain-level energy with high sampling frequency demonstrates the granularity supported by our monitoring (this experiment shows the sampling in 10Hz but Mojito/S supports the frequencies up to 100Hz with only a little overhead). While the client nodes show 15 clear energy spikes (one for each training round), the server node shows a different pattern.

Overall, the graph aligns with the expected behavior of each node: clients use more energy to train, while the server uses less energy and mainly shows memory activity during each round, and some steady CPU energy through the package domain. The results highlight the consistency of the measured values with the corresponding behavior of the nodes during training.

As shown in Fig. 4, I compare the power consumption recorded using RAPL and an external hardware power meter. From the original RAPL data, I sum the values from the package and DRAM domains and convert them to Watts (since the raw data was energy sampled every 0.1 seconds). For the power meter data, the sampling frequency was in Watts and measured every 1 second, so no further processing was needed.

Both sources show similar trends, which once again validates the correctness of our monitoring framework. However, the values from the power meter are higher than those from RAPL. This is because RAPL focuses on specific domains of processors, while the external power meter captures the entire device power, including other components outside RAPL’s scope.

Fig. 5 displays the number of packets and the amount of data (in bytes) received and transmitted by each node over time. The patterns are strongly correlated across nodes, reflecting the synchronized communication phases - one peak occurs when clients send their models to the server, and another when the server broadcasts the aggregated model back to the clients. This consistent traffic behavior across all clients demonstrates the communication-awareness of our framework. Based on these measurements, I can estimate the energy consumption associated with data transmission. Depending on the system context (*e.g.*, WAN), I can apply a suitable energy estimation model, such as the one referenced in [QPfM⁺23].

Fig. 6 shows the CPU time spent in user, system, and idle modes for each node. Each line in the graph represents a node, showing how its CPU resources were used during the entire training process. The plot clearly shows the difference between the client nodes and the server node. The server node shows consistently high idle time throughout the training process. This is expected, as the server does not perform local model training, and thus, it rarely utilizes CPU resources for user-level computations or intensive system operations. In contrast, the client nodes exhibit fluctuating CPU usage, with idle time decreasing significantly during training rounds. During these periods, the user time increases sharply, reflecting the intensive computations required for local model training, such as gradient calculation and weight updates. These tasks are CPU-bound and consume substantial processing power, resulting in a visible spike in user mode CPU usage. Although both systems are low, the client time is slightly higher than the server during the training phase due to data I/O, local resource overhead. Overall, this figure effectively illustrates the different workloads of each node.

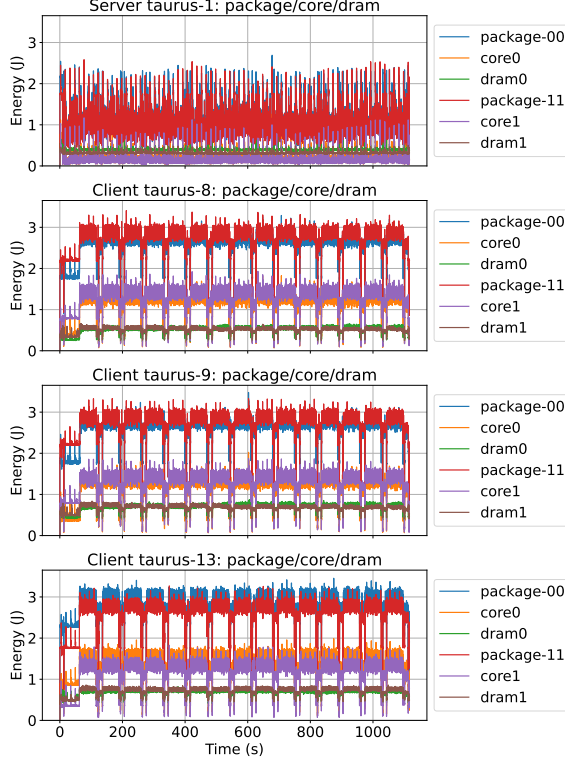


Figure 3: Energy consumption over time (1/10s intervals) on the server (taurus-1) and client nodes in the **CPU nodes** experiment, broken down by domain. Clients show clear peaks during training rounds; the server does not.

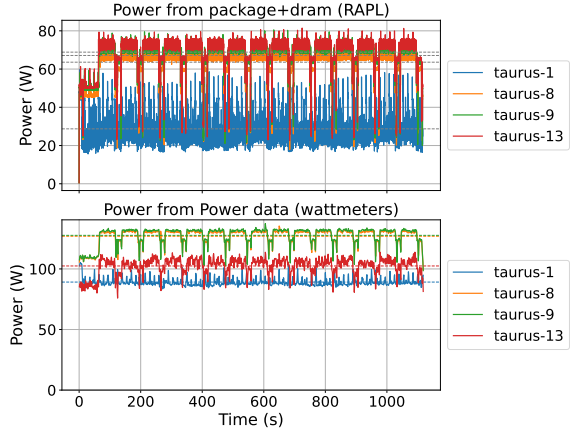


Figure 4: Comparison of power consumption measured by RAPL and an external meter in the **CPU nodes** experiment. Both align well, validating our framework. The meter shows higher values due to full-system measurement.

In the initial phase before training begins, Fig. 3, 4 and 5 all show patterns that differ from typical training rounds. In Fig. 3, 4, the client consumes energy but at a lower level than active training. Fig. 5 shows a noticeable amount of data transmission, highlighted by consistent peaks at the beginning and end of each training round, which correspond to the connection and communication phases. Fig. 6 also indicates higher idle time during this period. This behavior can be explained by the client waiting to connect, other clients joining, or the server issuing the command to start the training round.

Fig. 7 visualizes the performance and resource usage of different instances at four frequencies. The subplots analyze relationship between CPU frequencies and various performance (accuracy, execution time) and energy metrics (mean power, energy consumption and CO_2). The carbon intensity in France (31 g CO_2 eq/kWh), which is measured in grams of CO_2 equivalent per kilowatt-hour, is taken from the Electricity Map website¹¹.

The accuracy values shown are not intended for comparison but to illustrate the ability of experiment (the results are not statistically stable since random seeds are not fixed in the training process). As expected, execution time decreases with higher frequencies due to faster computation. In contrast, mean power consumption increases. Consequently, total energy consumption by a multiple of power and time duration also rises with frequency. The trends of the energy measured by both methods (Kwollect and Mojito/S) are consistent. However, the energy consumption measured by Kwollect are higher than those from Mojito/S because Kwollect measures the entire device, including all components, whereas Mojito/S only count the energy of package (CPU + iGPU), DRAM (RAM).

In addition, I observe that longer training (two epochs) generally leads to higher accuracy, as expected, but also results in increased execution time and energy consumption across all frequency settings. It highlights a basic but important trade-off between training depth and energy cost, especially

¹¹<https://app.electricitymaps.com/zone/FR/72h/hourly>

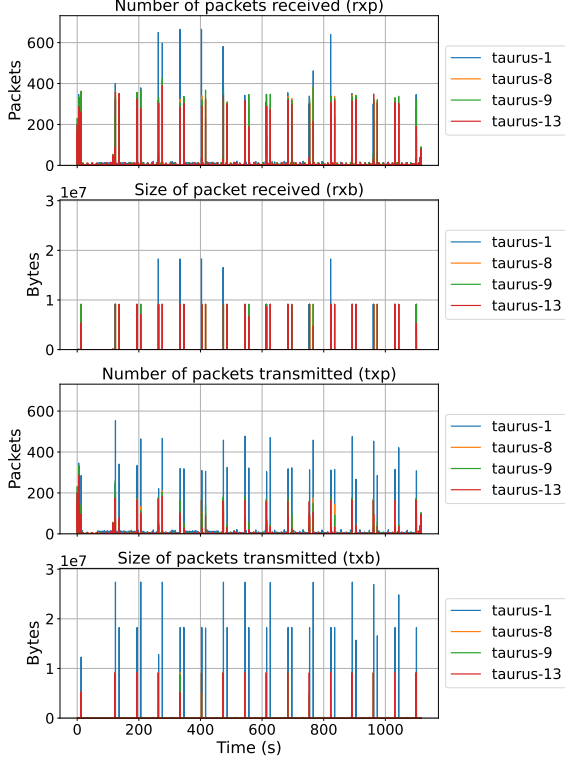


Figure 5: Communication traffic per node over time, measured in packets and bytes, during 1/10s intervals, in the **CPU nodes** experiment. Synchronized peaks reflect key FL phases, including model transmission.

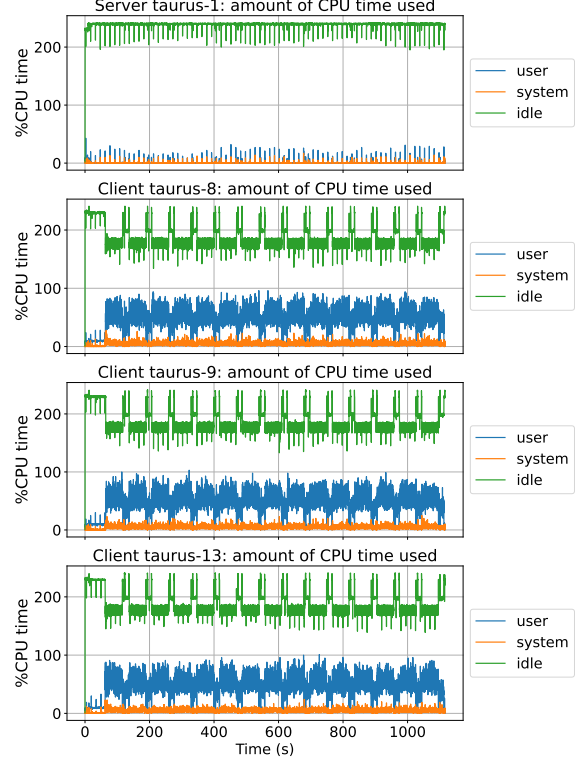


Figure 6: CPU usage distribution across server and clients in the **CPU nodes** experiment. Peaks show computational load on clients and coordination role of the server. Values above 100% are due to multi-core aggregation (2 processors $\times$ 12 cores).

when model performance gains are marginal.

B. Validating framework usability in large-scale

1. Experimental setup I executed this experiment on 20 machines from the **gros** cluster on the Grid’5000 testbed. These clusters were selected because it is a large resource with multiple node Intel CPUs with a feature of RAPL and the external power meter (PDU). List of used nodes: List of used nodes:

- Server node: **gros-16**.
- Client nodes: **gros-3**, **gros-22**, **gros-24**, **gros-[26-28]**, **gros-[29-31]**, **gros-[33-41]**, **gros-77**, **gros-91**.

Detailed specifications of these machines are as follows:

- Model: Dell PowerEdge R640.
- Manufacturing/arrival date: 2019-07-16 / 2019-09-04.
- CPU: Intel Xeon Gold 5220 (Cascade Lake-SP), x86_64, 2.20GHz, 1 CPU/node, 18 cores/CPU.
- Memory: 96 GB.
- Storage: 480 GB SSD SATA Micron MTFDDAK480TDN (on **gros-[1-26]**, **gros-[28-124]**) or 480 GB SSD SATA Samsung MZ7KH480HAHQ0D3 (on **gros-27**).

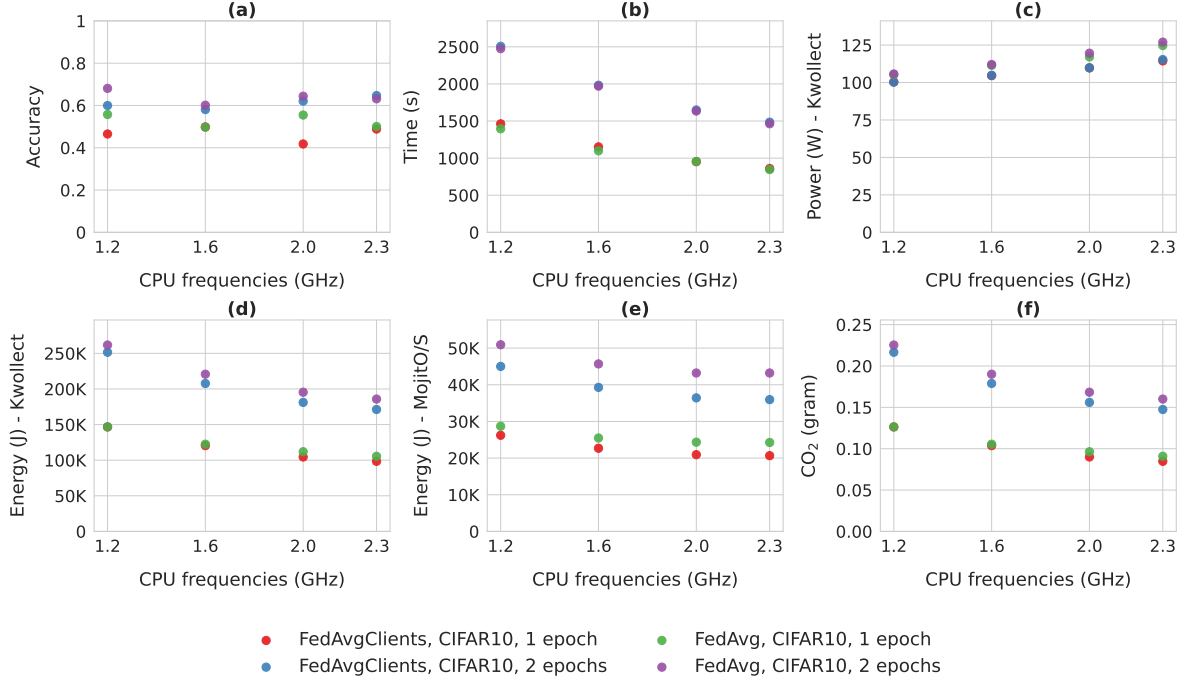


Figure 7: FL performance and resource consumption under different CPU frequency settings for 3 CPU clients in **CPU nodes** experiment.

(a) Accuracy after 15 rounds; (b) Processing time of FL; (c) Mean power consumption of entire devices, measured by Kwcollect; (d) Total energy consumption of entire devices, calculated by multiplying mean power (Kwcollect) with processing time; (e) Total energy consumption of two domains (Package + DRAM), measured by Mojito/S; (f) CO_2 emissions, estimated by multiplying energy consumption of entire devices with the carbon intensity in France (31 gCO₂eq/kWh). Each point represents the mean of two repetitions.

- Network: **eth0/eno1**, Ethernet, configured rate: 25 Gbps, model: Mellanox Technologies MT27710 Family [ConnectX-4 Lx], driver: **mlx5_core**, SR-IOV enabled.

The software dependency requirements are the same as those listed in Section 2.2.4.

I used the Flower framework with the **FedAvg** aggregation strategy, consisting of one server and 19 client nodes. Each client performed local training for one epoch. Other model settings, the number of rounds, and the dataset remain the same as those described in Table 1.

Based on the hardware conditions of **gros**, the list of chosen CPU frequencies in this experiment is [1.2, 1.6, 2, 2.2] GHz, which includes f_{min} , f_{max} , and two available frequencies in between.

2. Results analysis Fig. 8 illustrates the performance and resource consumption of one specific FL setup (**FedAvg**, CIFAR-10, one epoch) across four CPU frequency settings. The subplots analyze how frequency impacts various performance metrics (*e.g.*, accuracy, execution time) and energy-related metrics (*e.g.*, energy consumption measured in both ways).

The accuracy after 15 rounds remains low (around 0.1) due to the data distribution. The CIFAR-10 dataset is divided among 19 clients, so each client receives a smaller portion of the data than the first experiment with three clients. This limited local dataset makes it difficult for individual models to learn meaningful patterns and slows the global model’s convergence in the early training rounds.

The energy consumption trend in this experiment is also similar and consistent with the first one. However, since the hardware used in the two experiments is different, I do not draw any subjective conclusions from this. The main purpose of this second experiment is to confirm that our framework still works stably in large-scale scenarios.

From a usability standpoint, although adding more clients required additional configuration lines (approximately 4 lines per client, including ID, command, argument, IP address), the overall structure of the configuration file remained unchanged, demonstrating that the framework scales cleanly with minimal structural adjustments.

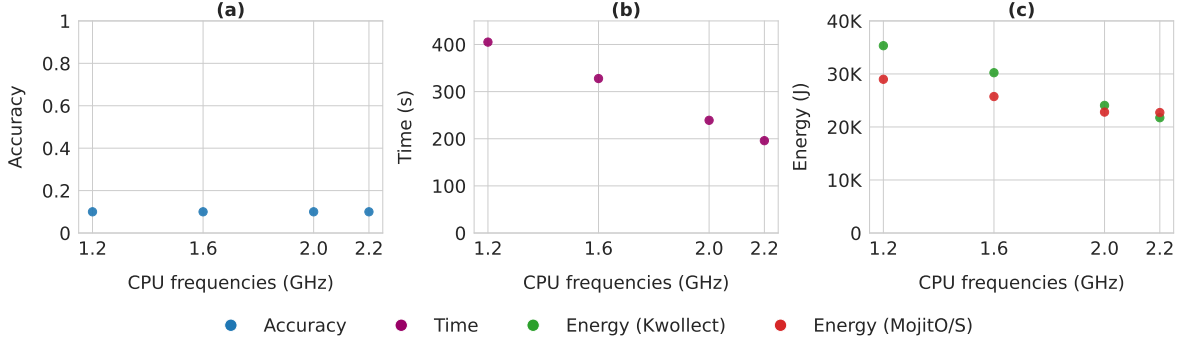


Figure 8: FL performance and resource consumption under different CPU frequency settings, shown for a single FL instance configuration (FedAvg, CIFAR-10, one local epoch), for 19 CPU clients in **large-scale** experiment.

(a) Accuracy after 15 rounds; (b) Processing time of FL; (c) Total energy consumption, computed in two ways: by multiplying mean power (from Kwollect) with processing time, and by summing energy across two domains (Package + DRAM) measured via Mojito/S. Each point represents a single experimental run.

C. Validating framework usability on GPU nodes

1. Experimental setup I executed this experiment on four machines from the **chiffnot** cluster on the Grid’5000 testbed. These clusters were selected because the nodes contain Intel CPUs with a feature of RAPL, and Nvidia GPUs with a feature of NVML. Besides, these nodes also are monitored by an external power meter (BMC). List of used nodes:

- Server node: **chiffnot-3**.
- Client nodes: **chiffnot-4**, **chiffnot-5**, **chiffnot-6**.

Detailed specifications of these machines are as follows:

- Model: Dell PowerEdge R740.
- Manufacturing/arrival date: 2018-07-17 / 2018-08-01.
- CPU: Intel Xeon Gold 6126 (Skylake-SP), x86_64, 2.60GHz, 2 CPUs/node, 12 cores/CPU.
- GPU: 2 × Nvidia Tesla P100-PCIE-16GB (16 GiB), compute capability: 6.0.
- Memory: 192 GB.
- Storage: 480 GB SSD SATA Intel SSDSC2KG480G7R.
- Network: **eth0/ens5f0**, Ethernet, configured rate: 25 Gbps, model: Intel Ethernet Controller XXV710 for 25GbE SFP28, driver: **i40e**.

The software requirements are the same as those listed in Section 2.2.4, except that TensorFlow should be installed with GPU support. I use `tensorflow[and-cuda]`, which includes the necessary CUDA and cuDNN libraries for GPU acceleration.

I used the Flower framework with the **FedAvg** aggregation strategy, consisting of one server and three client nodes. I evaluated two settings, with each client training locally for either one or two epochs per round. On each client, I selected only the first detected GPU for training. Other model settings, the number of rounds, and the dataset remain the same as those described in Table 1.

For this experiment, I selected the maximum GPU frequencies based on the hardware conditions of **chiffnot**, which is 1328 MHz.

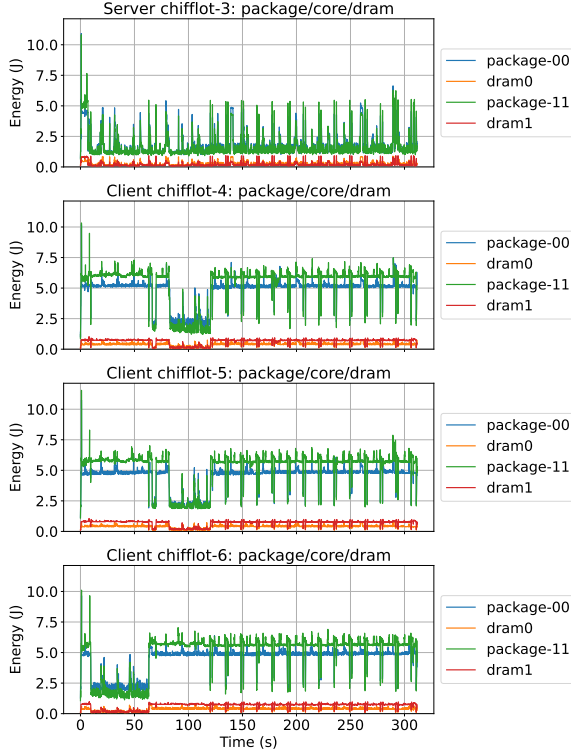


Figure 9: Energy consumption over time by domain (package, core, DRAM) in the **GPU nodes** experiment. Clients show clear peaks during training rounds; the server shows lower and more stable consumption.

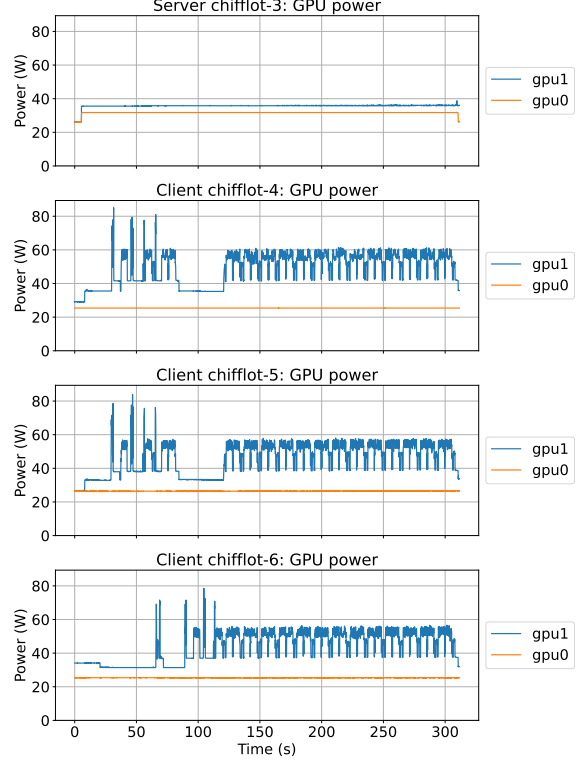


Figure 10: GPU power usage over time in the **GPU nodes** experiment. As with CPU domains, clients show distinct peaks during each training round; the server remains more stable.

2. Results analysis I provide the raw data visualization from one run of a specific setting (model aggregation: **FedAvg**, dataset: CIFAR-10, two local training epochs per round, GPU frequency: 1328 MHz). For the experiments performed on GPUs, in addition to the energy consumption measure from RAPL as shown in Fig 9, I also collect the GPU power consumption data from NVML as shown in Fig 10.

Fig 10 illustrates that, under stable conditions, each round consists of two consecutive power peaks corresponding to the two local epochs performed by each client. Specifically, a high power period is observed during the first epoch, followed by a short lower power period, and then another high power period for the second epoch, followed by a slightly longer lower power period before the next round begins. These consecutive peaks represent one full training round. In contrast, the server does not participate in the training process, so the server’s power graph is just a straight line without peaks.

In the first round, the data shows noticeable noise, with several short and high power peaks. This noise is typically attributed to system initialization and setup issues, such as the need to register CUDA libraries and load necessary resources.

Additionally, Fig 10 also shows that only two clients participated in the first round because client **chiffnot-6** has not yet established a connection with the server. This client only has its first peak at the end of the first global round, when the server sends an evaluate request to the clients.

In the following, Fig. 11 shows the comparison between the total energy consumption measured by RAPL/NVML and the power meter. The measuring device plugged into the **chiffnot** cluster node is BMC with a low sampling frequency (0.2 Hz or one sample every five seconds), which causes the graph to appear less smooth compared to the wattmeters used in the **taurus** cluster (Fig. 4). The power meter measures the entire device’s energy consumption, which is generally higher than the values recorded by RAPL/NVML.

Table 2 shows that increasing the number of local epochs from one to two leads to higher energy

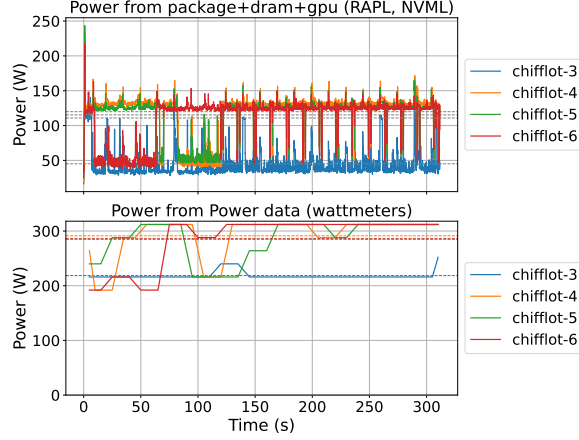


Figure 11: Comparison of power consumption measured by RAPL/NVML and an external power meter in **GPU nodes** experiment. The RAPL/NVML values include both package and DRAM domains, rescaled to 1-second intervals. The power meter readings are slightly higher as they measure full-system power, which goes beyond the scope of RAPL/NVML. The power pattern is not smooth due to the low sampling frequency of the BMC (0.2 Hz).

Table 2: Results of configurations: FedAvg, CIFAR-10, GPU frequency 1328 MHz, in two setting of one or two epoch of local training per round in **GPU nodes** experiment.

Number of epoch	1	2
Total energy by RAPL+NVML (J)	14525	20165
Total energy by wattmeters (J)	58643	80702
Accuracy	0.4903	0.6692
CO_2 (gram)	0.050	0.069

consumption, longer training time, and better accuracy, as expected from the logic of the **FedAvg** algorithm. Furthermore, the external power meter reports higher total energy because it measures the entire device. In addition, these consistent and reasonable results confirm that our FedEator framework is reliable.

2.3 Future work - 3rd year

My plan for the upcoming year includes the following:

- **Ensure reproducibility** of the framework by packaging the environment using Nix, allowing users to replicate experiments and results reliably.
- **Extend experiments** beyond the current simple setups by running tests with various FL strategies, datasets, and tasks from the literature or custom implementations, in order to evaluate the **trade-off between accuracy and energy consumption** (near future).
- Generalize the **environment module** to support custom node connection methods across different distributed systems (not limited to Grid’5000), aiming to improve the framework’s applicability and promote wider adoption (long-term goal).
- Once foundational trade-off results are available, I plan to address **real-world optimization and resource allocation**, by integrating optimization algorithms directly into the FL system along with energy measurement, to assess their effectiveness in practical deployments.
- **Last but not least:** submit publications and start writing the thesis.

3 Work situation

3.1 Difficulties encountered

There have been no major difficulties, and my work is progressing as planned. I am currently finalizing two manuscripts and preparing them for submission in the near future. In addition to that, intense summer heat and dealing with strong sunlight have been the only real challenges for me.

3.2 Research Outputs

- I have developed the **FedEator** framework, which is available as open-source here: <https://gitlab.irit.fr/sepia-pub/delight/eflwr>.
- I currently have two manuscripts under revision for resubmission: a **survey paper** and a paper describing the **FedEator framework**.
- I presented my work and had a paper accepted at the **COMPAS 2025** conference.

3.3 Teaching

I participated in teaching and giving tutorials for TP and TD sessions of the CSA class, specifically on **monitoring** and **MPI** in the *Parallel* course.

3.4 Training progress

- *Débat théâtral “Entre les lignes”* (10 April 2025), Toulouse III Paul Sabatier — 3 hours
- *Journée Doctorants ASR* (5 June 2024), IRIT Toulouse — 8 hours
- *MOOC: Reproducible Research II: Practices and tools for managing computations and data* (Inria) (25 June 2024 – 26 September 2024), Toulouse — 10 hours
- *MOOC: Recherche reproductible : principes méthodologiques pour une science transparente* (1 September 2023 – 22 July 2024), Online — 6 hours
- *Cours de Français Langue Étrangère – FLE* (2 September 2024 – 30 June 2025) — 23 hours (Grade: 14)
- *EcoICT Fall School 2024* (7 – 11 October 2024), IRISA, Beg Meil — 20 hours
- *What do we mean by Digital Sobriety?* (3 October 2024), IRIT Toulouse — 2 hours
- *PhDOOC – MOOC Doctorat et Poursuite de Carrière / PhD and Career Development* (1 October 2024 – 4 March 2025), PhDOOC platform — 12 hours

Total training hours: 84/100 hours

References

- [BTM⁺20] Daniel J Beutel, Taner Topal, Akhil Mathur, Xinchu Qiu, Javier Fernandez-Marques, Yan Gao, Lorenzo Sani, Kwing Hei Li, Titouan Parcollet, Pedro Porto Buarque de Gusmão, et al. Flower: A friendly federated learning research framework. *arXiv preprint arXiv:2007.14390*, 2020.
- [dC21] Georges da Costa. Mojito/S, November 2021.
- [Hid] Hidden for the peer-review process.
- [Int64] Intel Intel. and ia-32 architectures software developer’s manual. *Volume 3A: System Programming Guide, Part 1*(64):64, 64.

- [KHN⁺18] Kashif Nizam Khan, Mikael Hirki, Tapio Niemi, Jukka K Nurminen, and Zhonghong Ou. Rapl in action: Experiences in using rapl for power measurements. *ACM Transactions on Modeling and Performance Evaluation of Computing Systems (TOMPECS)*, 3(2):1–26, 2018.
- [KNW⁺23] Maximilian Kapsecker, Daniel N Nugraha, Christoph Weinhuber, Nicholas Lane, and Stephan M Jonas. Federated learning with swift: An extension of flower and performance evaluation. *SoftwareX*, 24:101533, 2023.
- [MHdBPM25] Andrew Mary Huet de Barochez, Stéphan Plassart, and Sébastien Monnet. Falafels: A tool for estimating federated learning energy consumption via discrete simulation. In *International Conference on Advanced Information Networking and Applications*, pages 141–152. Springer, 2025.
- [QPFM⁺23] Xinchu Qiu, Titouan Parcollet, Javier Fernandez-Marques, Pedro PB Gusmao, Yan Gao, Daniel J Beutel, Taner Topal, Akhil Mathur, and Nicholas D Lane. A first look into the carbon footprint of federated learning. *Journal of Machine Learning Research*, 24(129):1–23, 2023.
- [RCZ⁺20] Sashank Reddi, Zachary Charles, Manzil Zaheer, Zachary Garrett, Keith Rush, Jakub Konečný, Sanjiv Kumar, and H Brendan McMahan. Adaptive federated optimization. *arXiv preprint arXiv:2003.00295*, 2020.
- [RT24] Guillaume Raffin and Denis Trystram. Dissecting the software-based measurement of cpu energy consumption: a comparative analysis. *IEEE Transactions on Parallel and Distributed Systems*, 2024.
- [SRKB22] Stefano Savazzi, Vittorio Rampa, Sanaz Kianoush, and Mehdi Bennis. An energy and carbon footprint analysis of distributed and federated learning. *IEEE Transactions on Green Communications and Networking*, 7(1):248–264, 2022.